

1 实验目的与方法

1.1 词法分析器

实验一使用 JAVA 编程语言，在 JDK17 及以上版本的软件环境，实现了一个词法分析器，可以识别类 C 语言的保留字、标识符、常数、运算符和界符，并生成相对应的此法单元。本实验的核心是通过构建一个 DFA，根据输入源代码逐个字符解析，识别有效的单词并生成 Token 列表。

1.2 语法分析

实验二使用 JAVA 编程语言，在 JDK17 及以上版本的软件环境，在实验一的基础上新增了语法分析功能，可以根据在编译工作台生成的 grammar.txt（自定义文法规则），LR1_table.csv（编译工作台生成的 LR(1)分析表）以及实验一的词法分析器输出的 token 列表三份文件进行语法分析并生成 parser_list.txt 文件。

1.3 典型语句的语义分析及中间代码生成

实验三使用 JAVA 编程语言，在 JDK17 及以上版本的软件环境，在实验二的基础上新增了典型语句的语义分析和中间代码生成的功能。可以根据实验二生成的 parser_list.txt，已有的 grammar.txt, LR1_table.csv 和 token 表，对典型语句进行语义分析并生成中间代码。

1.4 目标代码生成

实验四使用 JAVA 编程语言，在 JDK17 及以上版本的软件环境，在实验三生成的中间代码基础上，新增了生成目标代码的功能。实验中采用了 RISC-V 语言作为目标汇编代码。通过对实验三生成的中间代码文件 intermediate_code.txt 进行解析，并结合实验二提供的 grammar.txt, LR1_table.csv 等文件，生成最终的 RISC-V 汇编代码文件 assembly_language.asm。

2 实验内容及要求

2.1 词法分析器

实验一的主要内容为实现一个类 C 语言词法分析器，主要功能为从输入文件中读取文件中的类 C 语言的代码，忽略无用符号，将源代码分解为正确的单词，并将其输出为二元组形式的 Token 列表。要求为正确识别类 C 语言中的标识符、关键字、常数、运算符和界符。

2.2 语法分析

实验二的主要内容为在实验一的基础之上实现语法分析功能，具体包括定义文法规则、生成 LR(1)分析表以及最终基于上述内容实现语法分析器，并将语法分析结果输出到 parser_list 中。要求为 parser_list 表中需要准确展示语法分析过程。

2.3 典型语句的语义分析及中间代码生成

实验三的主要内容为在实验二的基础上实现语义分析和中间代码生成功能。具体包括：

- 1.实现语义分析器 SemanticAnalyzer，用于检查程序中的变量声明、类型匹配和未声明变量的使用，确保输入代码符合语义规范；
- 2.实现中间代码生成器 IRGenerator，用于将输入的高级语言翻译为中间表示（IR），为后续目标代码生成做准备。

2.4 目标代码生成

实验四的主要内容为在实验三的基础之上实现生成 RISC-V 汇编代码的功能。具体内容包括：

1. 寄存器分配与管理：利用寄存器 t0-t6 及寄存器 a0，确保在执行过程中对寄存器资源的正确使用；
2. 汇编代码生成：根据中间代码的操作类型，包括加法、减法、乘法、赋值等，生成对应的 RISC-V 汇编指令，代码生成器在翻译中间代码时，识别每条指令的操作数和目标寄存器，生成 RISC-V 汇编指令。

3 实验总体流程与函数功能描述

3.1词法分析

3.1.1 编码表

词法分析器从 coding_map.csv 文件中读取保留字、运算符和分界符的编码规则，确保关键字和运算符可以被正确识别为相应的 Token 类型。

3.1.2 正则文法

词法分析中的正则文法定义了如何通过字符序列识别合法的单词：

1. 标识符：以字母或下划线开头，后面可以衔接字母、数字或下划线的字符序列，不可与关键字序列相同；
2. 整数常量：一个或多个数字组成的字符序列；
3. 运算符：预定义的符号，如+, -, *, /, =, ==等；

4. 关键字：由语言定义的保留字，如 `int`, `return`，这些字在语言中具有特定的含义

3.1.3 状态转换图

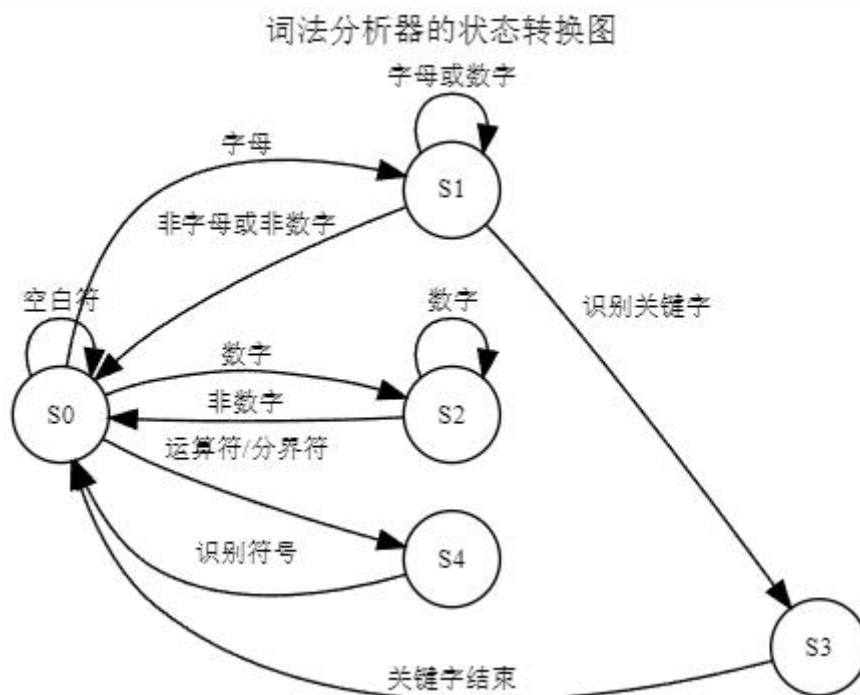


图 3.1.1 词法分析器的状态转换图

如上图，S0 为初始状态，S1 为读取标识符状态，S2 为读取整数常量状态，S3 为读取关键字状态，S4 为读取符号状态。

3.1.4 词法分析程序设计思路和算法描述

基于 DFA 设计词法分析程序，通过逐字符解析输入的源代码，根据字符类型进行状态转换，当解析到一个完整的单词时，将其转换为一个 Token 对象，并存入 Token 列表中。

初始状态：从输入文件的第一个字符开始进入初始状态，在初始状态下，根据字符的类型决定下一部操作；

标识符和关键字的识别：当遇到字母时，程序进入标识符/关键字识别状态，并继续读取字符，直到遇到非字母或非数字字符，识别完成后，程序检查单词是否为关键字，如果是关键字，则生成关键字 Token，否则生成标识符 Token 并存入符号表。

整数常量的识别：遇到数字时，程序进入整数常量识别状态并继续读取字符，直到遇到非数字字符，识别完成后生成整数常量 Token。

运算符和分界符：读取到运算符和分界符时，直接生成运算符和分界符的 Token。

符号表维护：识别到标识符时，程序将检查符号表中是否已经存在该标识符，如果

不存在，则将其加入符号表。

EOF 处理：所有字符都被解析完成后，程序生成一个表示文件结束的 EOF Token，并加入 Token 列表。

3.2 语法分析

3.2.1 拓展文法

拓展文法定义了非终结符、终结符机器对应的产生式，明确了如何从开始符号推导出合法的句子。实验中自定义文法规则内容在 `grammar.txt` 文件中。

3.2.2 LR1 分析表

LR(1)分析表存放在 `LR1_table.csv` 文件中，是 `grammar.txt` 文件在编译工作台中生成的图表，包含了语法分析过程中所需的移进（Shift）：将下一个输入符号移入符号栈，同时状态栈推入新的状态；规约（Reduce）：根据产生式，将符号栈中的若干个符号规约为非终结符，并根据 LR(1)表更新状态栈。

3.2.3 状态栈和符号栈的数据结构和设计思路

在 LR(1)语法分析器中：

状态栈：保存当前解析的状态，栈顶的状态决定下一个移进或规约操作，当发生移进时，新状态压入栈顶，当发生规约操作时，多个符号和状态出栈，根据 LR(1)表的指令更新新状态。

符号栈：保存已匹配的输入符号和非终结符，每次移进操作时，将输入符号压入栈顶；规约时，根据产生式将多个符号出栈并替换为非终结符。

3.2.4 LR 驱动程序设计思路和算法描述

程序会根据 `LR1_table` 中的状态转移规则，依次对输入的 Token 列表进行移进和规约操作，主要步骤为：

1. 初始化状态：状态栈和符号栈初始化为空栈，初始状态压入状态栈，输入 Token 列表准备；
2. 移进：读取下一个 Token 输入，查找当前状态栈顶状态与该 Token 对应的操作指令，如果是移进操作，将 Token 压入符号栈，同时将移进后的状态压入状态栈；
3. 规约：如果是规约操作，根据文法中的产生式，将符号栈中的若干个符号出栈，并将左侧的非终结符压入符号栈，同时根据 LR(1)表，使用状态栈更新状态；
4. 接受：如果 LR(1)表中的操作指令为接受，则说明输入符号列表匹配文法规则，语法分析成功；
5. 错误：如果遇到无法处理的输入符号或没有匹配的操作指令，程序报错。

3.3 语义分析和中间代码生成

3.3.1 翻译方案

在语法分析器的基础上，通过在规约阶段进行语义检查和生成中间代码，将输入的高级代码翻译为三地址码的中间表示，此翻译方案包括以下步骤：

1. 变量声明检查：在每次规约时，检查变量是否已经声明，确保未使用未声明的变量；
2. 表达式计算：将高级表达式分解为多个中间代码指令，如 MOV, ADD, SUB, MUL，使用临时变量存储中间结果。
3. 控制流处理：实现对简单的控制流语句的中间代码生成，确保后续可以生成目标代码。

3.3.2 语义分析和中间代码生成的数据结构

1. 符号表：用于存储标识符和标识符的类型信息，确保在使用标识符的时候已经进行了声明检查；
2. 操作数栈：用于存储表达式中的操作数，在生成中间代码时通过出栈和入栈操作维护计算顺序；
3. 临时变量生成器：用于生成中间代码中使用的临时变量，如 \$t0, \$t1 等。

3.3.3 语法分析程序设计思路和算法描述

1. 监听规约事件：通过在语法分析器中注册 SemanticAnalyzer 和 IRGenerator，监听每次规约操作，根据产生式对符号进行语义检查和中间代码生成。
2. 处理不同产生式：根据语法规则，在规约时触发不同的语义动作，生成相对应的中间代码；
3. 临时变量管理：在处理复杂表达式时，生成临时变量存储中间结果，并在生成 IR 时使用这些临时变量。

算法描述：

1. 语义分析：在每次规约 $D \rightarrow \text{int}$ 或 $S \rightarrow D \text{ id}$ 时，将标识符及其类型信息添加到符号表；在处理赋值语句 $S \rightarrow \text{id} = E$ 时，检查变量是否已声明，确保合法性。
2. 中间代码生成：在处理表达式 $E \rightarrow E + A$ 或 $E \rightarrow E - A$ 等产生式时，生成对应的中间代码指令，并将结果存入临时变量，使用栈操作维护表达式的计算顺序，确保操作数的正确性。

3.4 目标代码生成

3.4.1 设计思路和算法描述

1. 寄存器分配策略：使用 t_0-t_6 寄存器，通过动态分配与释放寄存器来管理变量，代码生成器在遍历中间代码时，为每个变量或中间结果分配寄存器，并跟踪器使用情况，具体包括：

- (1) 寄存器映射：每个操作数都需要分配一个寄存器；
- (2) 寄存器回收：寄存器不再被当前语句后续操作使用时，将其释放；
- (3) 溢出处理：若所有寄存器均被占用，则输出报错处理；

2. 汇编代码生成：

- (1) MOV 指令：将中间代码中的 MOV 操作翻译为 mv 指令，如果是立即数，则用 li 指令加载；
- (2) 算术运算：根据中间代码中的 ADD, SUB, MUL 指令，分别翻译为 RISC-V 汇编中的 add、sub、mul 指令；
- (3) RET 指令：将中间代码中的 RET 指令翻译为 mv a0, <寄存器>，确保返回值被正确加载到 a0 寄存器用于函数返回，然后执行 ret 指令结束函数；

- 3. 代码生成过程：汇编生成器按顺序遍历中间代码指令，通过查找操作数和结果变量对应的寄存器生成汇编代码，生成的汇编代码被存储到列表中，最终写入 assembly_language.asm 文件。

4 实验结果与分析

4.1 实验一的输入与输出

```
int result;  
int a;  
int b;  
int c;  
a = 8;  
b = 5;  
c = 3 - a;  
result = a * b - ( 3 + b ) * ( c - a );  
return result;
```

输入：

图 4.1.1 输入：input_code.txt

输出:

(int,)	(id,a)
(id,result)	(Semicolon,)
(Semicolon,)	(id,result)
(int,)	(=,)
(id,a)	(id,a)
(Semicolon,)	(*,)
(int,)	(id,b)
(id,b)	(-,)
(Semicolon,)	((),)
(int,)	(IntConst,3)
(id,c)	(+,)
(Semicolon,)	(id,b)
(id,a)	((),)
(=,)	(*,)
(IntConst,8)	((),)
(Semicolon,)	(id,c)
(id,b)	(-,)
(=,)	(id,a)
(IntConst,5)	((),)
(Semicolon,)	(Semicolon,)
(id,c)	(return,)
(=,)	(id,result)
(IntConst,3)	(Semicolon,)
(-,)	(\$,)

图 4.1.2 输出 token.txt

```
(a, null)
(b, null)
(c, null)
(result, null)
```

图 4.1.3 输出 old_symbol_table.txt

如上所示，词法分析器成功识别了输入代码中的关键字（int、return）、标识符（a、b...）、常数（3、8...）、运算符（-、=）和界符，并将其以正确的二元组格式输出到 token 和 old_symbol_table 文件中。

4.2 实验二的输入与输出


```

P -> S_list;
S_list -> S Semicolon S_list;
S_list -> S Semicolon;
S -> D id;
D -> int;
S -> id = E;
S -> return E;
E -> E + A;
E -> E - A;
E -> E & A;
E -> E / A;
E -> A;
A -> A * B;
A -> B;
B -> - B;
B -> ( E );
B -> id;
B -> IntConst;

```

输入：

4.2.1 输入 grammar.txt

```

状态,ACTION,,,,,,,,,GOTO,,,,,
,id,(,)+,-,*,/,&=,int,return,IntConst,Semicolon,$,S_list,S,D,E,A,B
0,shift 4,,,,,,,,,shift 5,shift 6,,,,,,,,1,2,3,,,
1,,,,,,,,,accept,,,,,
2,,,,,,,,,shift 7,,,,,
3,shift 8,,,,,,,,,
4,,,,,,,,,shift 9,,,,,
5,reduce D -> int,,,,,,,,,
6,shift 13,shift 14,,,,,shift 15,,,,,shift 16,,,,,10,11,12
7,shift 4,,,,,,,,,shift 5,shift 6,,,,,reduce S_list -> S Semicolon,17,2,3,,,
8,,,,,,,,,reduce S -> D id,,,,,
9,shift 13,shift 14,,,,,shift 15,,,,,shift 16,,,,,18,11,12
10,,,,,,,,,shift 19,shift 20,,shift 21,shift 22,,,,,reduce S -> return E,,,,,
11,,,,,reduce E -> A,reduce E -> A,shift 23,reduce E -> A,reduce E -> A,,,,,reduce E -> A,,,,,
12,,,,,reduce A -> B,reduce A -> B,reduce A -> B,reduce A -> B,reduce A -> B,reduce A -> B,,,,,reduce A -> B,,,,,
13,,,,,reduce B -> id,reduce B -> id,reduce B -> id,reduce B -> id,reduce B -> id,reduce B -> id,,,,,reduce B -> id,,,,,
14,shift 27,shift 28,,,,,shift 29,,,,,shift 30,,,,,24,25,26
15,shift 13,shift 14,,,,,shift 15,,,,,shift 16,,,,,31
16,,,,,reduce B -> IntConst,reduce B -> IntConst,reduce B -> IntConst,reduce B -> IntConst,,,,,reduce B -> IntConst,,,,,
17,,,,,,,,,reduce S_list -> S Semicolon S_list,,,,,
18,,,,,shift 19,shift 20,,shift 21,shift 22,,,,,reduce S -> id = E,,,,,
19,shift 13,shift 14,,,,,shift 15,,,,,shift 16,,,,,32,12
20,shift 13,shift 14,,,,,shift 15,,,,,shift 16,,,,,33,12

21,shift 13,shift 14,,,,,shift 15,,,,,shift 16,,,,,34,12
22,shift 13,shift 14,,,,,shift 15,,,,,shift 16,,,,,35,12
23,shift 13,shift 14,,,,,shift 15,,,,,shift 16,,,,,36
24,,,,,shift 37,shift 38,shift 39,,shift 40,shift 41,,,,,
25,,,,,reduce E -> A,reduce E -> A,reduce E -> A,shift 42,reduce E -> A,reduce E -> A,,,,,
26,,,,,reduce A -> B,reduce A -> B,reduce A -> B,reduce A -> B,reduce A -> B,reduce A -> B,,,,,
27,,,,,reduce B -> id,reduce B -> id,reduce B -> id,reduce B -> id,reduce B -> id,reduce B -> id,,,,,
28,shift 27,shift 28,,,,,shift 29,,,,,shift 30,,,,,43,25,26
29,shift 27,shift 28,,,,,shift 29,,,,,shift 30,,,,,44
30,,,,,reduce B -> IntConst,reduce B -> IntConst,reduce B -> IntConst,reduce B -> IntConst,reduce B -> IntConst,,,,,
31,,,,,reduce B -> - B,reduce B -> - B,reduce B -> - B,reduce B -> - B,reduce B -> - B,reduce B -> - B,,,,,
32,,,,,reduce E -> E + A,reduce E -> E + A,shift 23,reduce E -> E + A,reduce E -> E + A,,,,,
33,,,,,reduce E -> E - A,reduce E -> E - A,shift 23,reduce E -> E - A,reduce E -> E - A,,,,,
34,,,,,reduce E -> E / A,reduce E -> E / A,shift 23,reduce E -> E / A,reduce E -> E / A,,,,,
35,,,,,reduce E -> E & A,reduce E -> E & A,shift 23,reduce E -> E & A,reduce E -> E & A,,,,,
36,,,,,reduce A -> A * B,reduce A -> A * B,reduce A -> A * B,reduce A -> A * B,reduce A -> A * B,reduce A -> A * B,,,,,
37,,,,,reduce B -> ( E ),reduce B -> ( E ),reduce B -> ( E ),reduce B -> ( E ),reduce B -> ( E ),reduce B -> ( E ),,,,,,
38,shift 27,shift 28,,,,,shift 29,,,,,shift 30,,,,,45,26
39,shift 27,shift 28,,,,,shift 29,,,,,shift 30,,,,,46,26
40,shift 27,shift 28,,,,,shift 29,,,,,shift 30,,,,,47,26
41,shift 27,shift 28,,,,,shift 29,,,,,shift 30,,,,,48,26
42,shift 27,shift 28,,,,,shift 29,,,,,shift 30,,,,,49
43,,,,,shift 50,shift 38,shift 39,,shift 40,shift 41,,,,,
44,,,,,reduce B -> - B,reduce B -> - B,reduce B -> - B,reduce B -> - B,reduce B -> - B,reduce B -> - B,,,,,
45,,,,,reduce E -> E + A,reduce E -> E + A,reduce E -> E + A,shift 42,reduce E -> E + A,reduce E -> E + A,,,,,
46,,,,,reduce E -> E - A,reduce E -> E - A,reduce E -> E - A,shift 42,reduce E -> E - A,reduce E -> E - A,,,,,
47,,,,,reduce E -> E / A,reduce E -> E / A,reduce E -> E / A,shift 42,reduce E -> E / A,reduce E -> E / A,,,,,
48,,,,,reduce E -> E & A,reduce E -> E & A,reduce E -> E & A,shift 42,reduce E -> E & A,reduce E -> E & A,,,,,
49,,,,,reduce A -> A * B,reduce A -> A * B,reduce A -> A * B,reduce A -> A * B,reduce A -> A * B,reduce A -> A * B,,,,,
50,,,,,reduce B -> ( E ),reduce B -> ( E ),reduce B -> ( E ),reduce B -> ( E ),reduce B -> ( E ),reduce B -> ( E ),,,,,,

```

4.2.2 输入 LR1_label.csv

另外，实验二还接受如图 4.1.2 的实验一输出内容：token.txt（图 4.1.2）；

D -> int	E -> A
S -> D id	B -> id
D -> int	A -> B
S -> D id	E -> E + A
D -> int	B -> (E)
S -> D id	A -> B
D -> int	B -> id
S -> D id	A -> B
B -> IntConst	E -> A
A -> B	B -> id
E -> A	A -> B
S -> id = E	E -> E - A
B -> IntConst	B -> (E)
A -> B	A -> A * B
E -> A	E -> E - A
S -> id = E	S -> id = E
B -> IntConst	B -> id
A -> B	A -> B
E -> A	E -> A
B -> id	S -> return E
A -> B	S_list -> S Semicolon
E -> E - A	S_list -> S Semicolon S_list
S -> id = E	S_list -> S Semicolon S_list
B -> id	S_list -> S Semicolon S_list
A -> B	S_list -> S Semicolon S_list
B -> id	S_list -> S Semicolon S_list
A -> A * B	S_list -> S Semicolon S_list
E -> A	S_list -> S Semicolon S_list
B -> IntConst	S_list -> S Semicolon S_list
A -> B	P -> S_list

输出：

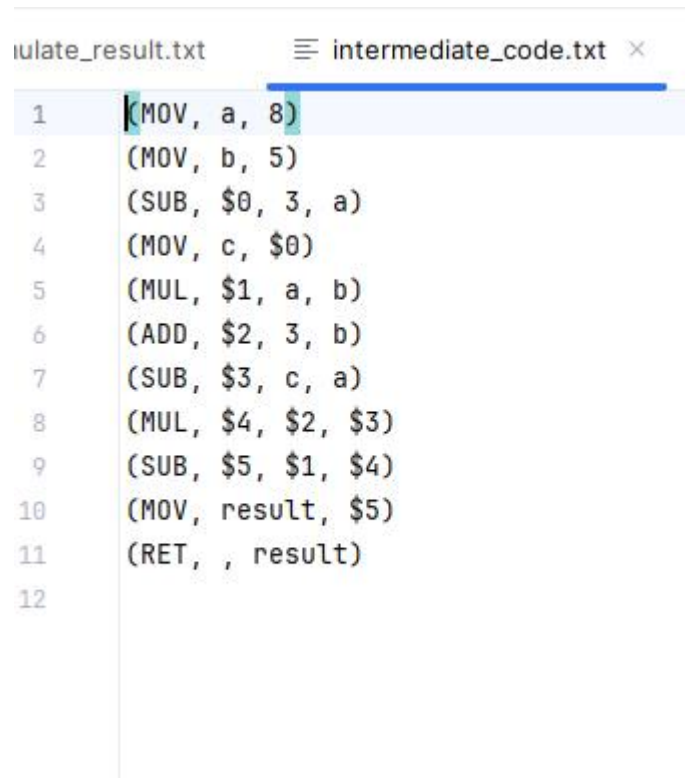
4.2.3 输出 parser_list.txt

如上所示，语法分析器成功读取了三份文档的输入内容，并将正确的语法分析内容输出到了 parser_list.txt 中存储。

4.3 实验三的输入与输出

实验三的输入包括实验一生成的 token 列表（图 4.1.2），实验二生成的 parser_list.txt(图 4.2.3)，grammar.txt(图 4.2.1)，LR1_table.csv（图 4.2.2），上文已有截图，此处不再赘述。

输出：



```
intermediate_code.txt
1 (MOV, a, 8)
2 (MOV, b, 5)
3 (SUB, $0, 3, a)
4 (MOV, c, $0)
5 (MUL, $1, a, b)
6 (ADD, $2, 3, b)
7 (SUB, $3, c, a)
8 (MUL, $4, $2, $3)
9 (SUB, $5, $1, $4)
10 (MOV, result, $5)
11 (RET, , result)
12
```

4.3.1 输出 intermediate_code.txt

C:\Users\admin\.jdk\openjdk-2:

语义分析器已成功接收

IR 生成完成, 已成功接受。

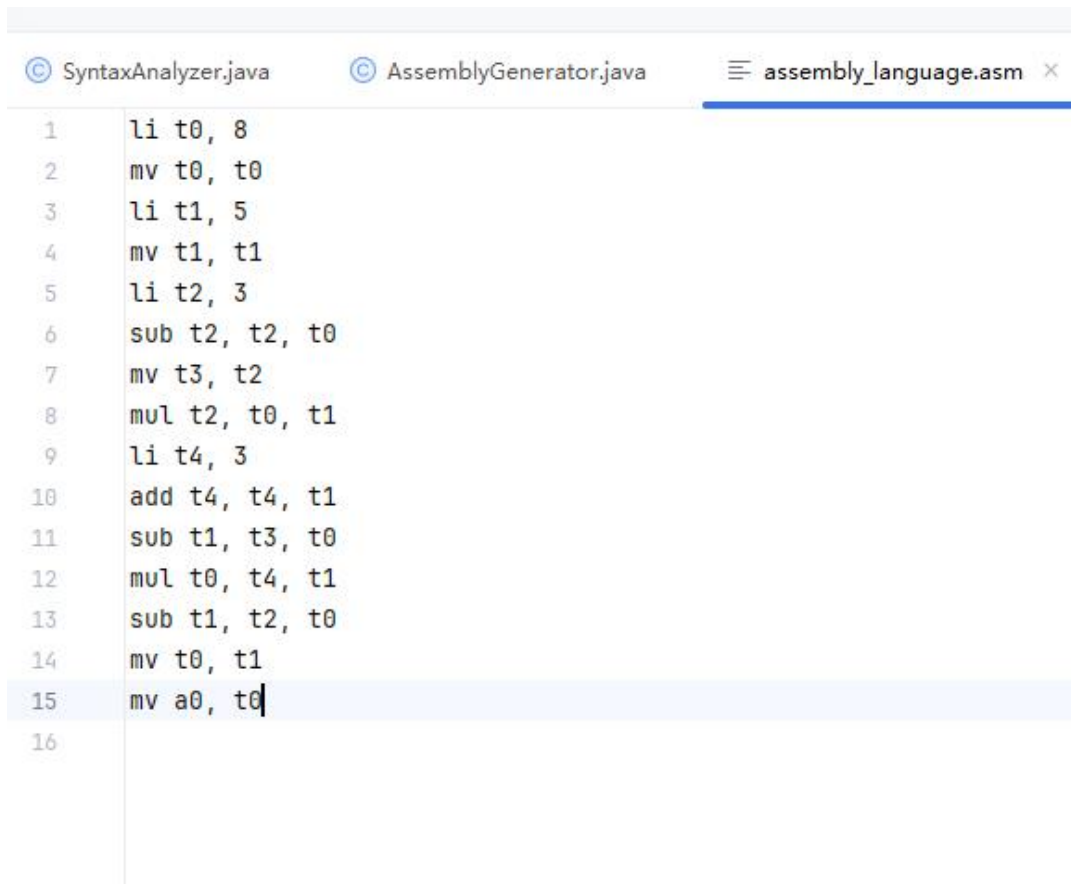
```
(MOV, a, 8)
(MOV, b, 5)
(SUB, $0, 3, a)
(MOV, c, $0)
(MUL, $1, a, b)
(ADD, $2, 3, b)
(SUB, $3, c, a)
(MUL, $4, $2, $3)
(SUB, $5, $1, $4)
(MOV, result, $5)
(RET, , result)
```

4.3.2 参考输出控制台打印信息

4.4 实验四输入与输出

实验四的输入为实验三生成的中间代码文件 intermediate_code.txt(图 4.3.1), 该文件包含了通过语义分析和中间代码生成器生成的中间代码, 上文已有截图, 此处不再赘述。最终输

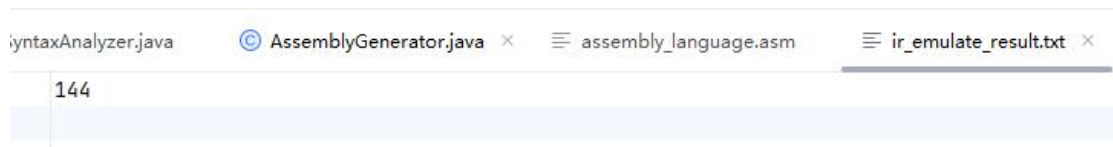
出 assembly_language.asm 文件（图 4.4.1），该文件包含翻译后的 RISC-V 汇编代码。
输出：



```
1  li t0, 8
2  mv t0, t0
3  li t1, 5
4  mv t1, t1
5  li t2, 3
6  sub t2, t2, t0
7  mv t3, t2
8  mul t2, t0, t1
9  li t4, 3
10 add t4, t4, t1
11 sub t1, t3, t0
12 mul t0, t4, t1
13 sub t1, t2, t0
14 mv t0, t1
15 mv a0, t0
16
```

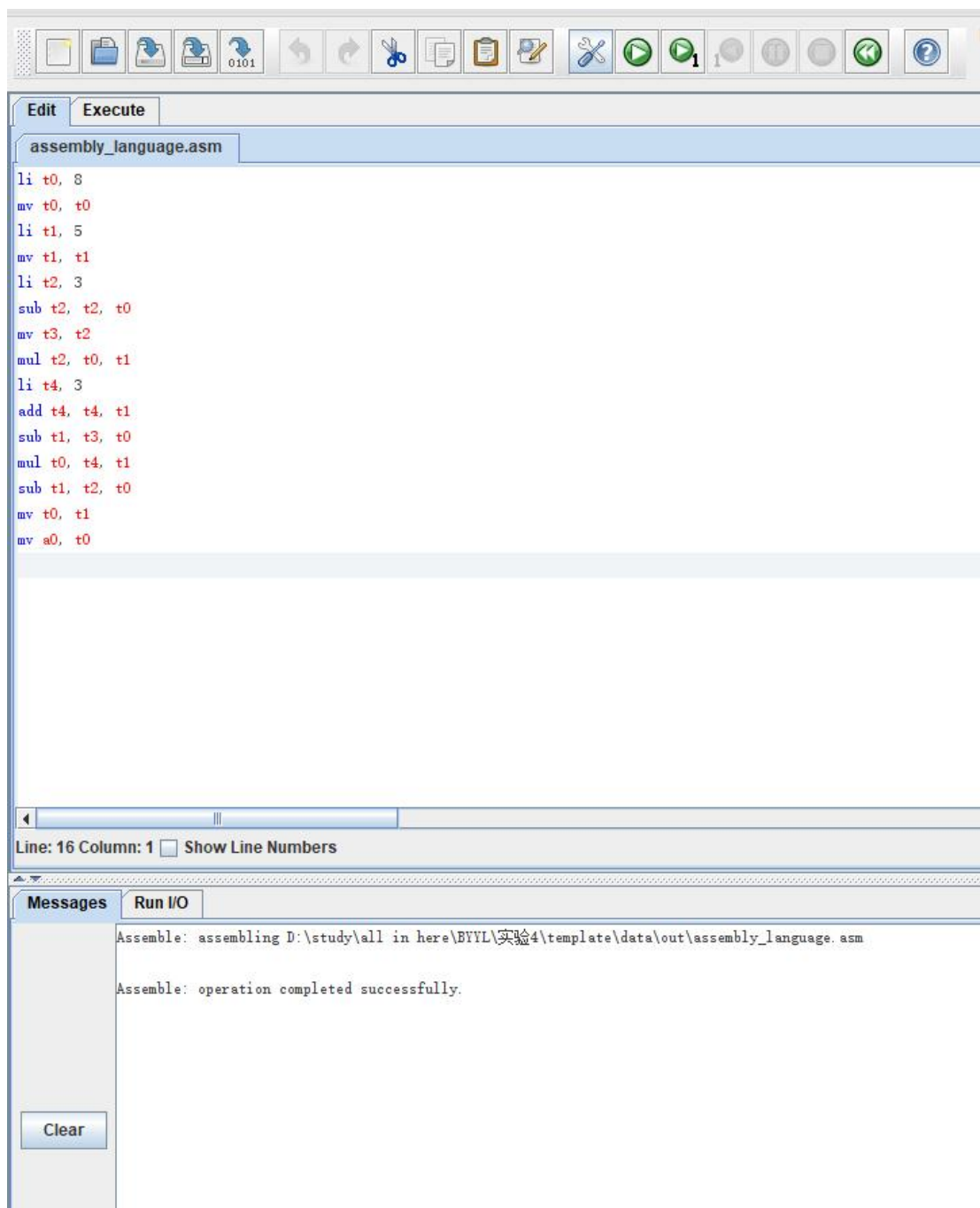
4.4.1 输出 assembly_language.asm

其中，输出内容的正确性可以通过 RARS(本实验使用的是 1.6 版本)运行验证，验证过程结果及结果如下图所示，其中，data/out/ir_emulate_result.txt 已经给出了参考答案，即变量 result 的最终数值：

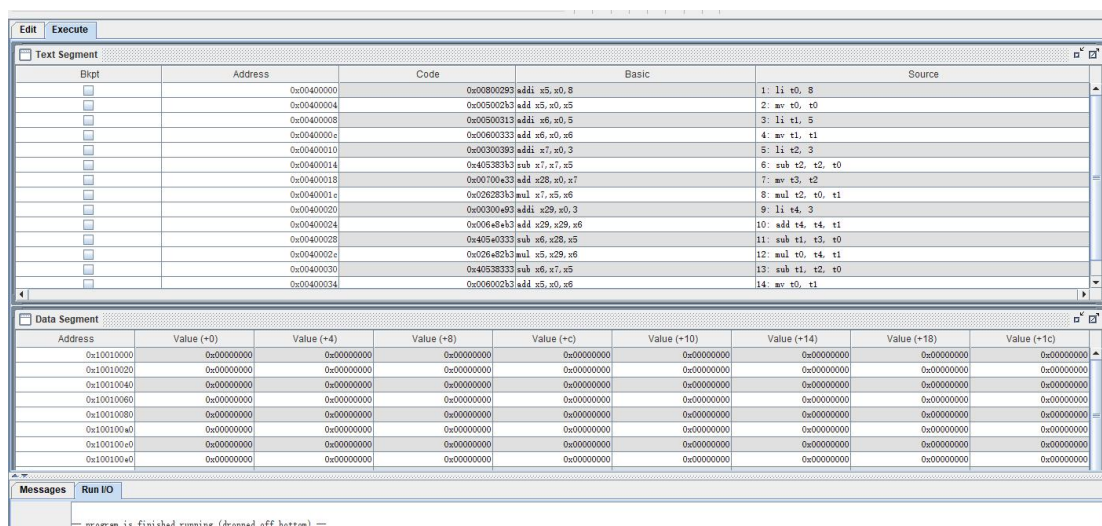


```
144
```

4.4.2 ir_emulate_result.txt



4.4.3 assembly_language.asm 导入到 RARS 中



4.4.4 assembly_language.asm 成功运行

zero	0	0x00000000
ra	1	0x00000000
sp	2	0x7fffffc
gp	3	0x10008000
tp	4	0x00000000
t0	5	0x00000090
t1	6	0x00000090
t2	7	0x00000028
s0	8	0x00000000
s1	9	0x00000000
a0	10	0x00000090
a1	11	0x00000000
a2	12	0x00000000
a3	13	0x00000000
a4	14	0x00000000
a5	15	0x00000000
a6	16	0x00000000
a7	17	0x00000000
s2	18	0x00000000
s3	19	0x00000000
s4	20	0x00000000
s5	21	0x00000000
s6	22	0x00000000
s7	23	0x00000000
s8	24	0x00000000
s9	25	0x00000000
s10	26	0x00000000
s11	27	0x00000000
t3	28	0xffffffffb
t4	29	0x00000008
t5	30	0x00000000
t6	31	0x00000000
pc		0x00400040

4.4.5 各寄存器运行后的数值

根据图 4.4.5 可知，最终运行结果，a0 的值为 0x00000090，换算为 10 进制为 144，与 ir_emulate_result.txt 内数值一致，证明了输出的正确。

5 实验中遇到的困难与解决办法

1. 语义分析中的重复声明检测：由于实验一与实验三完成的时间相距较远，实验一达到 `LexicalAnalyzer` 的 `run()` 方法已经将标识符 (`id`) 添加到了符号表中，故而在词法分析阶段已经将变量添加到了符号表中，实验三再次尝试将符号添加进符号表中时就会触发重复声明检测。解决方法是修改 `LexicalAnalyser`，去掉将标识符添加到符号表中的逻辑，仅保留标识符的识别和生成 `Token` 的部分。
2. 数字常量在语义分析中被视为未声明的标识符：处理赋值语句和表达式时，程序错误地将数字常量视为需要再符号表中声明的标识符，触发未声明标识符报错。因为在 `whenReduce` 方法中，对表达式操作数进行符号表检查时没有区分标识符和常量，程序视图在符号表中查找数字常量导致报错。解决方法是添加对常量的判断逻辑，确保数字常量不会进行符号表查找。
3. 寄存器分配与管理复杂：由于只能使用有限的寄存器 `t0-t6`，在处理复杂的表达式时，寄存器的数量可能需要合理规划，需要有效分配与释放寄存器，在实验过程中，变量 `b` 所使用的寄存器 `t5` 就曾在计算过程中被覆盖为 `0`，导致最终结果错误。解决方法是在寄存器分配和释放时，特别关注关键变量的生命周期，确保它们不再被后续指令使用时才释放其寄存器。同时避免在中间计算过程中覆盖这些关键寄存器。

实验收获：通过四次实验帮助我了解了编译器的各个阶段如何逐步实现，也提醒我注意到整个项目各个阶段之间的关联性，如实验三编写时出现了由于实验一考虑不周导致输出报错的情况。

实验建议：可以提供更多测试样例，如增加复杂表达式或嵌套结构的盐工，以测试生成的汇编代码在更多场景下的正确性，如实验四在进行汇编代码生成时，完全可以指定变量 `a,b,c,result` 对应的寄存器而非动态规划寄存器，即便指定寄存器也可以通过测试。